

Authentification par OTP

Middleware d'erreurs, génération d'OTP, register / login / refresh / mot de passe oublié

Projet : **mkp_localhost** | Branche de départ : **auth**

À rendre pour le jeudi 24 septembre 2026

Objectifs

À la fin de ce TP, vous devez être capable de :

- centraliser la gestion des erreurs d'une API Express avec un middleware dédié ;
- modéliser une table **Otp** dans Prisma, avec la **raison** de l'OTP, et écrire la migration ;
- générer, stocker et vérifier un code OTP à usage unique avec la bibliothèque **otplib** ;
- implémenter un cycle d'authentification complet : inscription + vérification du compte, connexion, renouvellement du token, mot de passe oublié / réinitialisation, le tout appuyé sur les OTP.

Rappel du projet existant

Stack : Express 5, TypeScript, Prisma 7 (générateur prisma-client, adapter @prisma/adapters-pg), PostgreSQL, Zod 4, bcrypt. Lancement : `npm run dev`.

```
src/  
  app.ts                -> instance Express (express.json())  
  server.ts             -> prisma.$connect() puis app.listen()  
  config/prisma.ts      -> client Prisma singleton  
  modules/auth/  
    auth.controller.ts   -> register (incomplet)  
    auth.routes.ts       -> VIDE  
    auth.middlewares.ts  -> VIDE  
    schemas/register.schema.ts -> schéma Zod (non branché)  
prisma/  
  schema.prisma         -> modèle User uniquement
```

Étape 0 — Corriger le code existant (obligatoire)

Avant d'ajouter des fonctionnalités, corrigez les défauts suivants dans `auth.controller.ts`, `server.ts` et `config/prisma.ts`. Pour chacun, expliquez en une phrase dans votre README **pourquoi** c'était un bug.

- `bcrypt.hash()` est asynchrone : il manque un `await`.
- Le mot de passe (haché) est renvoyé au client dans la réponse de `register`.
- `import { success } from "zod"` : import inexistant à supprimer.
- `require("bcrypt")` dans un fichier TS : utiliser `import bcrypt from "bcrypt"` et installer `@types/bcrypt`.
- Le schéma Zod `registerSchema` n'est jamais utilisé (`req.body as any`).
- `server.ts` affiche le port 3000 en dur alors qu'il écoute sur `PORT`.
- `import 'dotenv'` dans `config/prisma.ts` ne charge rien : utiliser `import 'dotenv/config'` (et le placer en premier).

TP 1 — Middleware de gestion des erreurs

Objectif : aucune route ne doit gérer elle-même le format des erreurs. Tout passe par `next(error)` et un middleware final unique.

1.1 Classe `AppError`

Créer `src/utils/AppError.ts` :

```
export class AppError extends Error {
  constructor(
    public message: string,
    public statusCode: number = 500,
    public details?: unknown,
  ) {
    super(message);
    this.name = "AppError";
  }
}
```

1.2 Middleware de validation `Zod`

Dans `src/modules/auth/auth.middlewares.ts`, écrire une fabrique `validate(schema)` qui valide `req.body` avec `schema.safeParse()`. En cas d'échec, transmettre l'erreur à `next()` (ne pas répondre directement). En cas de succès, remplacer `req.body` par les données validées.

```
import { NextFunction, Request, Response } from "express";
import { ZodType } from "zod";

export const validate =
  (schema: ZodType) => (req: Request, _res: Response, next: NextFunction) => {
    const result = schema.safeParse(req.body);
    if (!result.success) return next(result.error); // ZodError
    req.body = result.data;
    next();
  };

```

1.3 Middleware d'erreur global

Créer `src/middlewares/error.middleware.ts`. Il doit avoir la signature à **4 paramètres** (`err`, `req`, `res`, `next`) et produire toujours le même format :

```
{ "success": false, "message": "...", "errors": [...] } // errors optionnel
```

Cas à traiter, dans cet ordre :

Type d'erreur	Statut HTTP	Contenu attendu
ZodError	400	message « Validation failed » + tableau <code>errors</code> [{ path, message }] (utiliser <code>err.issues</code>)
AppError	<code>err.statusCode</code>	<code>err.message</code> (+ details si présent)
Prisma P2002 (contrainte unique)	409	« A record with this value already exists » + le champ concerné (<code>err.meta.target</code>)
Prisma P2025 (introuvable)	404	« Resource not found »
JsonWebTokenError / TokenExpiredError	401	« Invalid or expired token »

Type d'erreur	Statut HTTP	Contenu attendu
Toute autre erreur	500	« Internal server error ». En NODE_ENV=development, ajouter stack ; en production, ne jamais exposer le détail.

Ajouter aussi un middleware **404** (route inconnue) juste avant le middleware d'erreur. Dans `app.ts`, l'ordre doit être : routes, puis 404, puis gestion d'erreur.

```
app.use("/api/auth", authRoutes);
app.use(notFound);
app.use(errorHandler); // toujours en dernier
```

Note : Express 5 transmet automatiquement à `next()` les promesses rejetées d'un handler async. Le `try/catch { next(error) }` reste accepté mais n'est plus obligatoire : expliquez dans le README le choix que vous faites.

TP 2 — Table Otp et génération d'OTP

2.1 Nouveau modèle Prisma

Un OTP n'a de sens que pour **une raison précise** : le code envoyé pour vérifier un e-mail ne doit pas pouvoir servir à réinitialiser un mot de passe. La raison est donc portée par une énumération et fait partie de la clé de recherche. Ajouter dans `prisma/schema.prisma` :

```
enum OtpReason {
  EMAIL_VERIFICATION // validation du compte après register
  PASSWORD_RESET     // mot de passe oublié
  LOGIN_2FA          // bonus : connexion en deux étapes
}

model Otp {
  id      String    @id @default(uuid())
  code    String    // HASH du code, jamais le code en clair
  reason  OtpReason
  expiresAt DateTime
  usedAt   DateTime? // null tant que non consommé
  attempts Int       @default(0) // tentatives de vérification échouées

  userId  String
  user    User      @relation(fields: [userId], references: [id], onDelete: Cascade)

  createdAt DateTime @default(now())

  @@index([userId, reason])
}
```

Compléter le modèle User avec la relation inverse `otps Otp[]`, puis créer la migration :

```
npx prisma migrate dev --name add_otp_table
```

Joindre le fichier `migration.sql` généré au rendu et vérifier qu'il contient bien le type "OtpReason", la clé étrangère et l'index.

2.2 Utilitaire OTP — `src/utils/otp.ts`

Installer la bibliothèque : `npm i otplib`. L'utilitaire doit vivre dans `src/utils/otp.ts` et exposer **au minimum** les fonctions suivantes (signatures imposées) :

```

import { totp } from "otplib";
import bcrypt from "bcrypt";
import prisma from "../config/prisma";
import { OtpReason } from "../generated/prisma/enums";

export const OTP_TTL_MINUTES = 10;
export const OTP_MAX_ATTEMPTS = 5;

/** Génère un code numérique à 6 chiffres via otp lib. */
export function generateOtpCode(): string { /* totp.generate(secret) */ }

/**
 * Crée un OTP pour (userId, reason) :
 * - invalide les OTP précédents non utilisés de même (userId, reason)
 * - stocke le HASH du code + expiresAt = now + OTP_TTL_MINUTES
 * - retourne le code EN CLAIR (à envoyer à l'utilisateur, jamais stocké)
 */
export async function createOtp(userId: string, reason: OtpReason): Promise<string> {}

/**
 * Vérifie un code pour (userId, reason). Lève une AppError (400/410/429) si :
 * - aucun OTP actif, ou déjà utilisé (usedAt != null)
 * - expiré (expiresAt < now)
 * - attempts >= OTP_MAX_ATTEMPTS
 * - code incorrect -> incrémenter attempts avant de lever l'erreur
 * En cas de succès : positionner usedAt = now.
 */
export async function verifyOtp(userId: string, reason: OtpReason, code: string): Promise<void> {}

```

Indications sur otp lib :

- générer un secret aléatoire par appel (`authenticator.generateSecret()` ou `crypto.randomBytes`) puis `totp.generate(secret)` ; configurer `totp.options = { digits: 6 }` ;
- comme le code est stocké haché avec `bcrypt`, la vérification se fait avec `bcrypt.compare(code, otp.code)` et non avec `totp.check()` ;
- l'envoi réel (e-mail / SMS) n'est **pas** demandé : écrire un `src/utls/mailler.ts` avec une fonction `sendOtp(email, code, reason)` qui fait un `console.log`. Bonus : `nodemailer` + `Mailtrap/Ethereal`.

Note : Le code OTP en clair ne doit apparaître **ni** en base, **ni** dans une réponse HTTP. Il n'existe que dans le log de `sendOtp` (dev) ou dans l'e-mail.

TP 3 — Contrôleurs d'authentification

Installer `npm i jsonwebtoken` et `npm i -D @types/jsonwebtoken`. Ajouter à `.env` et `.env.example` :

```

JWT_ACCESS_SECRET=changeme_access
JWT_REFRESH_SECRET=changeme_refresh
JWT_ACCESS_EXPIRES=15m
JWT_REFRESH_EXPIRES=7d

```

Créer `src/utls/jwt.ts` avec `signAccessToken(userId)`, `signRefreshToken(userId)`, `verifyAccessToken(token)` et `verifyRefreshToken(token)`. Chaque route possède son schéma Zod dans `modules/auth/schemas/` et est branchée avec `validate(...)`. Toutes les routes sont préfixées par `/api/auth`.

Routes à livrer

Méthode & route	Body	Comportement attendu
POST /register	first_name, last_name, email, phone_number, password	409 si email ou téléphone déjà pris (une seule requête findFirst avec OR). Hache le mot de passe, crée l'utilisateur (isVerified=false), createOtp(user.id, EMAIL_VERIFICATION), sendOtp. 201 avec l'utilisateur sans le champ password.
POST /verify-email	email, code	verifyOtp(user.id, EMAIL_VERIFICATION, code) puis isVerified=true. 200 « Account verified ».
POST /resend-otp	email, reason	Regénère et renvoie un OTP pour la raison donnée. Toujours 200, même si l'e-mail est inconnu (ne pas révéler l'existence d'un compte).
POST /login	email, password	401 « Invalid credentials » si utilisateur inconnu ou mot de passe faux (même message). 403 si isVerified=false. 200 avec { accessToken, refreshToken, user }.
POST /refresh-token	refreshToken	Vérifie le refresh token avec JWT_REFRESH_SECRET, vérifie que l'utilisateur existe encore, renvoie 200 { accessToken } (nouveau). 401 si invalide/expiré.
POST /forgot-password	email	Si l'utilisateur existe : createOtp(user.id, PASSWORD_RESET) + sendOtp. Réponse toujours 200 « If this email exists, a code has been sent ».
POST /reset-password	email, code, new_password	verifyOtp(user.id, PASSWORD_RESET, code), hache et enregistre le nouveau mot de passe. 200 « Password updated ». Bonus : invalider les refresh tokens existants.
GET /me	— (header Authorization: Bearer <accessToken>)	Middleware authenticate dans auth.middlewares.ts : 401 sans token valide, sinon 200 avec l'utilisateur courant (sans password).

Exigences transversales

- Format de réponse unique : { success, message, data? } en succès, { success:false, message, errors? } en erreur (via le middleware du TP 1).
- Aucun res.status(4xx) dans les contrôleurs : lever une AppError et laisser le middleware répondre.
- Aucune requête Prisma en dehors des contrôleurs et de src/utils/otp.ts.
- Le champ password ne sort **jamais** d'une réponse (utiliser select ou omit de Prisma).
- Type strict : pas de any, npm run typecheck doit passer.

Scénario de test attendu (ordre)

Fournir un fichier requests.http (extension REST Client de VS Code) ou une collection Postman qui rejoue ce scénario. Le code OTP est lu dans la console du serveur.

```

1. POST /register          -> 201, OTP EMAIL_VERIFICATION dans la console
2. POST /login            -> 403 (compte non vérifié)
3. POST /verify-email     -> 200
4. POST /verify-email     -> 400/410 (le même code ne marche plus : usedAt renseigné)
5. POST /login            -> 200, accessToken + refreshToken
6. GET /me                -> 200 avec le Bearer accessToken
7. POST /refresh-token    -> 200, nouvel accessToken
8. POST /forgot-password  -> 200, OTP PASSWORD_RESET dans la console
9. POST /reset-password   -> 200 (essayer avec le code EMAIL_VERIFICATION : doit échouer)
10. POST /login (ancien mdp) -> 401 ; avec le nouveau -> 200
11. POST /register (même email) -> 409
12. POST /register (body vide) -> 400 avec le tableau errors de Zod

```

Livrables et modalités de rendu

- Une branche `tp-auth-<prenom-nom>` créée depuis `auth`, poussée sur le dépôt, avec une **Pull Request vers main** ouverte avant **jeudi 24 septembre 2026, 23h59**.
- Des commits réguliers et explicites (au minimum un par TP : erreurs, otp, contrôleurs).
- Un `README.md` : installation, variables d'environnement, tableau des routes, explication des bugs de l'Étape 0, choix techniques (try/catch vs Express 5, TTL, nombre de tentatives...).
- Le fichier `requests.http` ou la collection Postman exportée.
- Le fichier `.env` ne doit **jamais** être commité ; `.env.exemple` doit être à jour.

Arborescence cible

```

src/
  app.ts
  server.ts
  config/prisma.ts
  middlewares/
    error.middleware.ts      (errorHandler, notFound)
  utils/
    AppError.ts
    otp.ts                  (generateOtpCode, createOtp, verifyOtp)
    jwt.ts
    mailer.ts               (sendOtp -> console.log)
  modules/auth/
    auth.routes.ts
    auth.controller.ts      (register, verifyEmail, resendOtp, login,
                             refreshToken, forgotPassword, resetPassword, me)
    auth.middlewares.ts     (validate, authenticate)
    schemas/
      register.schema.ts    login.schema.ts    verify-otp.schema.ts
      refresh.schema.ts     forgot-password.schema.ts  reset-password.schema.ts
  prisma/
    schema.prisma           (User + Otp + enum OtpReason)
    migrations/xxxx_add_otp_table/
  requests.http
  README.md

```

Barème (20 points)

Critère	Points
Étape 0 : bugs corrigés et expliqués	2
TP 1 : AppError, validate(), errorHandler (Zod / AppError / Prisma / JWT / 500), notFound, ordre dans app.ts	4
TP 2 : modèle Otp + enum OtpReason + migration ; utilitaire src/utls/otp.ts complet (hash, TTL, usedAt, attempts, invalidation des anciens)	5
TP 3 : register / verify-email / resend-otp / login / refresh-token / forgot-password / reset-password / me	6
Qualité : typage strict, format de réponse uniforme, aucune fuite de password ni d'OTP, README, requests.http, commits	3
Bonus : LOGIN_2FA (login en deux étapes par OTP), envoi réel via nodemailer, révocation des refresh tokens en base	+2

Questions de réflexion (à traiter dans le README)

- Pourquoi stocke-t-on le **hash** de l'OTP et non le code lui-même, alors qu'il expire en 10 minutes ?
- Pourquoi /forgot-password et /resend-otp répondent-ils 200 même quand l'e-mail n'existe pas ?
- Que se passerait-il si le champ reason n'existait pas dans la table Otp ?
- Pourquoi limiter le nombre de tentatives (attempts) sur un code à 6 chiffres ?
- Quelle différence entre invalider un access token et invalider un refresh token ? Lequel est possible sans base de données ?

Bon courage. Posez vos questions sur le canal du cours avant mercredi soir.